Agentes adversarios duales LSTM–GAN para la detección no supervisada de comportamientos maliciosos en registros de servidores universitarios

Arturo Ruiz Tarazona
Facultad de Ingeniería Eléctrica y Electrónica (FIEE), Escuela Profesional de Ingeniería de Ciberseguridad
Universidad Nacional de Ingeniería (UNI), Lima, Perú
Curso de Inteligencia Artificial II (CBS06), semestre 2026-2

Los registros (*logs*) de servidores universitarios crecen a un ritmo que impide su revisión manual y carecen de etiquetas que permitan entrenar detectores supervisados. Este estudio evalúa si un modelo de agentes adversarios duales de reconstrucción latente, un autocodificador LSTM y un discriminador secuencial que compiten en un juego *minimax* (LSTM–GAN), mejora la detección de comportamientos maliciosos frente a Isolation Forest, técnica no supervisada de referencia. Se empleó un diseño cuasi-experimental con medida pre-prueba/post-prueba sobre datos reales crudos: 3 217 544 eventos de PostgreSQL, GitLab y un ERP en Odoo, agrupados en 53 416 ventanas de cinco minutos mediante Drain3 y Word2Vec. Los modelos se entrenaron sin etiquetas y se evaluaron con etiquetas débiles derivadas de reglas de seguridad, con partición cronológica y diez semillas por subconjunto (30 pares). El LSTM–GAN incrementó el puntaje F1 de 25.06 % a 32.26 % ($p < .001$, $r = .85$) y redujo la tasa de falsos positivos de 6.27 % a 2.88 % ($p = .002$, $r = .58$), sin diferencia significativa en el área bajo la curva ROC (86.52 % frente a 85.68 %; $p = .529$). El análisis de ablación mostró que el autocodificador sin componente adversarial alcanza un desempeño igual o superior y más estable. Se concluye que la reconstrucción temporal de series semánticas es eficaz para reducir falsas alarmas sin datos etiquetados, mientras que la contribución del discriminador debe justificarse empíricamente en cada entorno.

Palabras clave: detección de anomalías, registros del sistema, autocodificador LSTM, red generativa adversarial, aprendizaje no supervisado, ciberseguridad

Ideas destacadas

- Un autocodificador LSTM con discriminador adversarial (LSTM–GAN) detecta inyecciones SQL y escaneos web en registros universitarios reales sin usar etiquetas.
- Frente a Isolation Forest, el LSTM–GAN eleva el F1 en 7.2 puntos y reduce las falsas alarmas en un 54 %, con significancia estadística en 30 pares.
- El componente adversarial no supera al autocodificador LSTM aislado, que obtiene la mayor capacidad discriminante (AUC de 89.04 %).

1. Introducción

Los sistemas informáticos registran de forma continua su actividad en archivos de *logs*, que constituyen la principal fuente de evidencia para detectar fallos, usos indebidos e intrusiones. El volumen de estos registros en infraestructuras modernas hace inviable su inspección manual, y las reglas o firmas escritas a mano solo reconocen escenarios previstos de antemano, por lo que se requieren técnicas que aprendan automáticamente el comportamiento normal y señalen sus desviaciones (Landauer et al., 2023). En una universidad, un mismo conjunto de servidores aloja bases de datos académicas y administrativas, plataformas de control de versiones y aplicaciones de gestión expuestas a Internet, de modo que una intrusión en cualquiera de ellos compromete información sensible y la continuidad de los servicios.

La variable dependiente de este estudio es la *detección de comportamientos maliciosos*, cuya problemática presenta tres rasgos. Primero, las anomalías son raras y no se conocen de antemano, por lo que rara vez existen etiquetas para entrenar clasificadores supervisados; obtenerlas a escala industrial es costoso y debe repetirse cuando cambia el comportamiento normal (Albert, 2025). Segundo, los métodos no supervisados tienden a producir un número elevado de falsos positivos, lo que satura a los operadores y reduce la confianza en las alertas (Zamanzadeh Darban et al., 2024). Tercero, buena parte de los resultados publicados se obtienen sobre unos pocos conjuntos de referencia previamente

etiquetados (HDFS, BGL, Thunderbird, OpenStack), con supuestos optimistas como entrenar solo con datos normales o ajustar umbrales sobre el propio conjunto de prueba (Albert, 2025; Landauer et al., 2023).

Las arquitecturas basadas en reconstrucción ofrecen una alternativa sin etiquetas: un autocodificador aprende a reconstruir el comportamiento normal y las entradas que no logra reconstruir se consideran anómalas (Zamanzadeh Darban et al., 2024). Su combinación con redes generativas adversariales (GAN), en las que un discriminador obliga al codificador a producir representaciones latentes más fieles, ha mostrado mejoras en series temporales industriales y de Internet de las cosas (Gong et al., 2023; Qian et al., 2025). Sin embargo, **el vacío** que aborda este trabajo es la ausencia de evidencia sobre el desempeño de estos agentes adversarios duales en registros *crudos y heterogéneos* de infraestructura universitaria real, sin etiquetas, con partición cronológica y comparados estadísticamente con una técnica tradicional.

El **propósito** del estudio es evaluar en qué medida un modelo LSTM–GAN de reconstrucción latente mejora la detección de comportamientos maliciosos sin etiquetas en registros de servidores universitarios. Los **objetivos específicos** son: (OE1) determinar su efecto sobre la exactitud de la detección (F1); (OE2) determinar su efecto sobre la tasa de falsas alarmas (FPR); y (OE3) determinar su efecto sobre la capacidad discriminante (AUC-ROC), en todos los casos frente a Isolation Forest. En coherencia, se plantea la hipótesis general de que el modelo LSTM–GAN mejora significativamente la detección y las hipótesis específicas H1 (aumenta el F1), H2 (reduce la FPR) y H3 (aumenta el AUC). La **motivación** es práctica: los equipos de seguridad universitarios disponen de abundantes registros pero no de analistas para etiquetarlos. La **importancia** radica en aportar un experimento reproducible con datos reales, que cuantifica el efecto del modelo y de su componente adversarial mediante ablación. El **aporte** incluye además una representación de ventanas de cinco minutos como series semánticas que corrige la debilidad de la reconstrucción secuencial ante ráfagas de ataque repetitivas.

2. Revisión de la literatura

Landauer et al. (2023) revisaron sistemáticamente 62 enfoques de aprendizaje profundo para detectar anomalías en *logs*. Identificaron que las redes recurrentes son las más usadas, que los autocodificadores y las GAN son las opciones naturales para el aprendizaje no supervisado y que casi todas las evaluaciones dependen de cuatro conjuntos de datos con anomalías de tipo secuencial. Advirtieron, además, que heurísticas simples alcanzan resultados competitivos en HDFS. Por ello recomendaron evaluar con datos más diversos que incluyan artefactos de ciberataques y no solo fallos.

Albert (2025) demostró que los métodos semi-supervisados del estado del arte colapsan cuando el conjunto de entrenamiento contiene anomalías mezcladas, condición habitual en datos industriales sin depurar. También mostró que la división cronológica, frente a la aleatoria, reduce drásticamente el F1 reportado: en BGL, de 92.7 % a 42.6 % con DeepLog. Esto evidencia que la literatura sobreestima el desempeño esperable en producción.

En el plano arquitectónico, Gong et al. (2023) propusieron un autocodificador recurrente asistido por una GAN. En él, el discriminador distingue la codificación latente de la entrada de la recodificación de su reconstrucción, y el conjunto superó a métodos clásicos en datos industriales. Qian et al. (2025) integraron autocodificadores y GAN en RGAnomaly y combinaron la pérdida de reconstrucción con la de discriminación en una puntuación única de anomalía. Ambos trabajos se validaron con series de sensores, no con registros de texto.

Nasirzadeh et al. (2025) alcanzaron un F1 medio de 99.61 % en siete conjuntos de *logs* de sistemas operativos, pero con un enfoque supervisado que requiere etiquetas. En síntesis, la literatura evidencia el potencial de los modelos adversariales de reconstrucción, pero no responde si aportan valor frente a técnicas tradicionales en registros universitarios reales, sin etiquetas y evaluados con rigor cronológico. Ese es el vacío que aborda este estudio.

3. Metodología

3.1. Enfoque, tipo y diseño

El estudio es cuantitativo, de tipo aplicado y de nivel explicativo. Se adoptó un diseño cuasi-experimental de comparación con medida pre-prueba/post-prueba sobre las mismas unidades. La pre-prueba corresponde al desempeño de la técnica tradicional (Isolation Forest), el estímulo es la implementación del modelo LSTM–GAN y la post-prueba es el desempeño de este último sobre idénticas ventanas de evaluación. Se eligió este diseño porque permite atribuir las diferencias al modelo y no a la muestra, y porque contrastar un modelo propuesto con líneas de base sobre los mismos datos es la práctica habitual del área (Landauer et al., 2023; Nasirzadeh et al., 2025). Se añadió un tercer tratamiento de ablación (LSTM-AE sin discriminador) para aislar la contribución del componente adversarial.

3.2. Descripción del conjunto de datos real

Se utilizaron registros *crudos* del repositorio institucional *uni-logs*, publicado por el docente (https://uni-logs.us-mia-1.linodeobjects.com/list.html). Se analizaron tres de sus cuatro subconjuntos, que se describen en la Tabla 1:

- `logdb`: servidor PostgreSQL 12.
- `loggit`: plataforma GitLab, con registros de acceso de nginx, `sshd` y `gitlab-shell`.

- `logsapp`: ERP CubicERP basado en Odoo, con registros del servidor principal, del servidor *gevent* y de gunicorn.

Se excluyeron los registros de instalación del sistema operativo (`dpkg`, `apt`, `bootstrap`) por no reflejar la operación del servicio. La población la constituye la totalidad de eventos registrados; la muestra, obtenida por conveniencia, comprende 3 217 544 eventos agrupados en 53 416 ventanas.

3.3. Técnicas e instrumentos de recolección

La técnica fue el análisis documental de registros digitales. Los instrumentos fueron tres:

- El analizador de plantillas Drain3, que separa la parte constante de cada mensaje de sus parámetros variables mediante un árbol de profundidad fija (Nasirzadeh et al., 2025; Zhao et al., 2025).
- Una ficha de reglas de seguridad para generar etiquetas débiles de referencia (Tabla 2; Apéndice C).
- Los guiones reproducibles en Python (PyTorch, scikit-learn, gensim), que registran cada corrida.

El etiquetado por reglas o términos indicativos tiene precedente en Nasirzadeh et al. (2025). Es pertinente aquí porque los registros no incluyen etiquetas. Las etiquetas *no* se usan para entrenar ningún modelo, solo para evaluarlos.

3.4. Procedimiento experimental

El protocolo se ejecutó en tres fases automatizadas.

- **Fase de preparación.** Se leyeron los registros, se convirtieron todas las marcas de tiempo a UTC y se unieron las líneas de continuación de trazas y sentencias SQL. Las líneas de acceso web se codificaron como *método–primer segmento de ruta–extensión–estado HTTP–familia de agente*, para que Drain3 conservara la señal de ataque. Después, Drain3 se aplicó en línea, en orden cronológico, y los eventos se agruparon en ventanas fijas de cinco minutos, estrategia recomendada cuando no existen identificadores de sesión (Landauer et al., 2023).
- **Partición.** Cada subconjunto se dividió cronológicamente en 60 % para entrenamiento, 20 % para validación y 20 % para prueba. Así se evita la fuga de información futura señalada por Albert (2025). El conjunto de entrenamiento *no* se depuró, por lo que contiene 64 ventanas maliciosas; el escenario es, por tanto, genuinamente no supervisado.
- **Pre-prueba y estímulo.** Isolation Forest, LSTM-AE y LSTM–GAN se entrenaron sobre las mismas ventanas, con diez semillas aleatorias por subconjunto.
- **Post-prueba.** El umbral de decisión ρ de cada modelo se fijó en el percentil 95 de sus puntuaciones de validación, sin intervención del tramo de prueba (Albert, 2025; Qian et al., 2025), y se calcularon los indicadores sobre el tramo de prueba.

3.5. Indicadores y análisis estadístico

La exactitud se midió con el puntaje F1, la precisión y la exhaustividad; las falsas alarmas, con la FPR y la especificidad; y la capacidad discriminante, con el AUC-ROC y el coeficiente de correlación de Matthews (MCC) (Almodovar et al., 2024; Zamanzadeh Darban et al., 2024; Zhang et al., 2025):

$$F1 = \frac{2PR}{P+R}, \qquad FPR = \frac{FP}{FP+TN}. \tag{1}$$

El AUC-ROC es independiente del umbral, lo que lo hace adecuado para comparar puntuaciones continuas (Gorokhov et al., 2023). Para cada indicador se calcularon la media, la desviación estándar, la diferencia absoluta $\Delta = \text{post} - \text{pre}$ y la variación relativa.

La prueba de Shapiro–Wilk rechazó la normalidad de las diferencias en F1, FPR y AUC ($p < .05$). Por ello, las hipótesis se contrastaron con la prueba de rangos con signo de Wilcoxon para muestras relacionadas, con $\alpha = .05$, sobre 30 pares (10 semillas $\times$ 3 subconjuntos) y por subconjunto (10 pares), con tamaño del efecto $r = Z/\sqrt{n}$. Como análisis complementario, se estimó mediante *bootstrap* (2 000 remuestreos) el intervalo de confianza al 95 % de la diferencia de AUC a nivel de ventana.

3.6. Comparación con metodologías similares

El preprocesamiento sigue la cadena análisis de plantillas, representación semántica y agrupación por ventanas de Zhao et al. (2025) y Zhang et al. (2025), que también emplean Drain y Word2Vec. El criterio de anomalía por error de reconstrucción coincide con Aslam et al. (2024) y Gong et al. (2023). La puntuación que combina reconstrucción y discriminación procede de Qian et al. (2025).

El estudio se diferencia de esos trabajos en tres aspectos: usa datos crudos sin depurar en lugar de conjuntos de referencia etiquetados, aplica una partición estrictamente cronológica y somete el resultado a pruebas de significancia y a ablación, elementos poco frecuentes en la literatura revisada (Landauer et al., 2023).

4. Desarrollo

Los modelos se organizan jerárquicamente en tres niveles de complejidad creciente. El primero es Isolation Forest, puntual y sin contexto temporal. El segundo es el LSTM-AE, que añade contexto temporal y reconstrucción. El tercero es el LSTM–GAN, que añade el discriminador adversarial. La Figura 1 resume el flujo.

Tabla 1

Caracterización de los subconjuntos de registros crudos analizados

Subconjunto	Fuente	Eventos	Plantillas	Ventanas	Periodo (UTC)	Ent./Val./Prueba	Maliciosas (E/V/P)
`logdb`	PostgreSQL 12	267 896	186	5 719	01/03/2026 – 12/05/2026	3 431 / 1 144 / 1 144	2 / 1 / 90
`loggit`	GitLab	56 135	121	3 398	30/10/2025 – 12/05/2026	2 038 / 680 / 680	38 / 28 / 21
`logsapp`	CubicERP (Odoo)	2 893 513	361	44 299	20/11/2025 – 12/05/2026	26 579 / 8 860 / 8 860	24 / 2 / 8
Total		3 217 544	668	53 416		32 048 / 10 684 / 10 684	64 / 31 / 119

Nota. Ventanas de cinco minutos con al menos un evento. Las ventanas maliciosas corresponden a etiquetas débiles (Tabla 2), usadas solo para evaluar. E = entrenamiento; V = validación; P = prueba.

Tabla 2

Reglas de seguridad para el etiquetado débil por ventana

Subconjunto	Una ventana se etiqueta como maliciosa si …
`logdb`	contiene ≥ 1 firma de inyección SQL (`sleep(`, `XOR(`, `OR n=n`, `UNION SELECT`) o ≥ 3 fallos de sondeo (base de datos o rol inexistente, autenticación fallida).
`loggit`	contiene ≥ 1 URL con carga de ataque (SQLi/XSS, *path traversal*, `.php`, `wp-*`, `.env`), ≥ 1 agente escáner, ≥ 3 fallos SSH o ≥ 10 respuestas HTTP 4xx.
`logsapp`	contiene ≥ 1 URL con carga de ataque o ≥ 10 respuestas HTTP 4xx.

Nota. Se excluye el código 499 (cierre de conexión por el cliente) porque no indica una petición errónea.

4.1. Representación semántica de las ventanas

Cada plantilla k se representa mediante el promedio normalizado $\mathbf{e}_k \in \mathbb{R}^{300}$ de los vectores Word2Vec (*skip-gram*) de sus términos, entrenados sobre el vocabulario de plantillas sin usar etiquetas (Zhao et al., 2025). Cada ventana t con conteos $c_{t,k}$ se representa como

$$\mathbf{x}_t = \sum_k \log(1 + c_{t,k})\, \mathbf{e}_k, \tag{2}$$

estandarizado con la media y la desviación del tramo de entrenamiento. La entrada del modelo es la serie $X_t = (\mathbf{x}_{t-W+1}, \ldots, \mathbf{x}_t)$ con $W = 20$ (100 minutos) y paso 1, de modo que cada ventana recibe una puntuación propia.

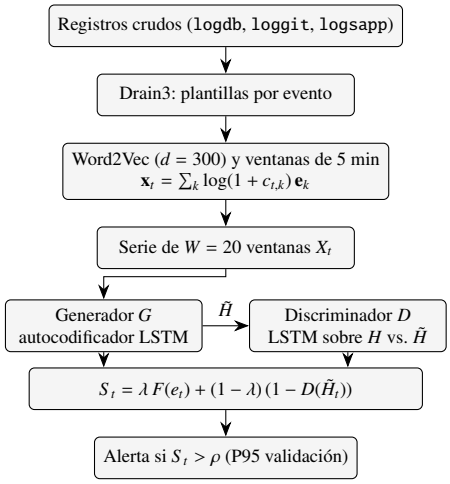
Esta decisión corrige un problema detectado en pruebas preliminares. Al modelar secuencias de *eventos*, las ráfagas de ataque, compuestas por cientos de peticiones idénticas, resultaban trivialmente reconstruibles y obtenían puntuaciones bajas.

4.2. Línea de base: Isolation Forest

Isolation Forest aísla observaciones mediante particiones aleatorias recursivas: las anomalías requieren menos particiones para quedar aisladas. Es una referencia no supervisada habitual en este dominio (Landauer et al., 2023; Nasirzadeh et al., 2025). Se usaron 200 árboles sobre el vector $\log(1+c_{t,k})$ de cada ventana, y la puntuación de anomalía es la puntuación de aislamiento con signo invertido.

Figura 1

Arquitectura del modelo LSTM–GAN de agentes adversarios duales



Nota. Elaboración propia. $F(\cdot)$ es la función de distribución empírica del error de reconstrucción en validación.

4.3. Generador: autocodificador LSTM

El codificador LSTM recorre la serie y la resume en el estado oculto final $\mathbf{z}_t = \mathbf{h}_W \in \mathbb{R}^{128}$:

$$\mathbf{h}_i = \mathrm{LSTM}_{enc}(\mathbf{x}_i, \mathbf{h}_{i-1}). \tag{3}$$

El decodificador recibe $\mathbf{z}_t$ en cada paso y reconstruye la serie, $\hat{\mathbf{x}}_i = W_o\, \mathrm{LSTM}_{dec}(\mathbf{z}_t, \mathbf{s}_{i-1}) + \mathbf{b}_o$. La pérdida de reconstrucción es el error cuadrático medio sobre las posiciones válidas m_i:

$$\mathcal{L}_{rec} = \frac{1}{\sum_i m_i} \sum_{i=1}^{W} m_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2. \tag{4}$$

Tabla 3

Configuración de los modelos

Parámetro	Valor
Dimensión Word2Vec / estado oculto LSTM	300 / 128
Ventanas por serie (W) / paso	20 / 1
Unidades del discriminador	64
α (peso de $\mathcal{L}_{rec}$) / λ (peso en S_t)	0.8 / 0.7
Optimizador / tasa de aprendizaje	Adam / 10^{-3}
Tamaño de lote / máx. secuencias por época	1 024 / 8 192
Épocas máximas / paciencia (parada temprana)	30 / 5
Árboles de Isolation Forest	200
Umbral ρ	P95 de validación
Semillas por subconjunto	10

Nota. El límite de 8 192 secuencias por época solo afecta a `logsapp`.

4.4. Discriminador adversario secuencial

Siguiendo a Gong et al. (2023), el discriminador D es una LSTM de 64 unidades con salida sigmoide. Recibe la secuencia de estados ocultos $H = \mathbf{h}_{1:W}$ de la entrada real y la secuencia $\tilde{H} = \text{LSTM}_{enc}(\hat{X})$ obtenida al recodificar la reconstrucción, y aprende a distinguirlas. El entrenamiento alterna dos pasos:

$$\mathcal{L}_D = -\Big[\log D(H) + \log\big(1 - D(\tilde{H})\big)\Big], \tag{5}$$

$$\mathcal{L}_G = \alpha\,\mathcal{L}_{rec} + (1-\alpha)\Big[-\log D(\tilde{H})\Big], \tag{6}$$

con $\alpha = 0.8$. La ecuación (6) obliga al generador a producir reconstrucciones cuya codificación latente sea indistinguible de la real.

4.5. Puntuación dual y decisión

Para cada ventana se combinan el error de reconstrucción de su última posición, $e_t = \|\mathbf{x}_t - \hat{\mathbf{x}}_t\|_2^2$, y el rechazo del discriminador (Qian et al., 2025):

$$S_t = \lambda\, F_{val}(e_t) + (1-\lambda)\Big(1 - D(\tilde{H}_t)\Big), \tag{7}$$

donde F_{val} es la función de distribución empírica del error en validación, que lleva ambos términos a la escala $[0, 1]$, y $\lambda = 0.7$. La ventana se clasifica como maliciosa si $S_t > \rho$. El LSTM-AE de ablación usa $S_t = F_{val}(e_t)$ y se entrena solo con la ecuación (4). La Tabla 3 resume la configuración.

5. Resultados

5.1. Desempeño por modelo y subconjunto

La Tabla 4 y la Figura 2 presentan los indicadores en el tramo de prueba. En el consolidado de los tres subconjuntos, el LSTM–GAN obtuvo F1 = 32.26 %, FPR = 2.88 % y AUC = 85.68 %. Isolation Forest obtuvo F1 = 25.06 %, FPR = 6.27 % y AUC = 86.52 %. Esto equivale a ΔF1 = +7.19 puntos porcentuales (+28.7 %) y ΔFPR = −3.39 puntos (−54.0 %). El LSTM-AE obtuvo F1 = 33.56 %, FPR = 3.28 % y AUC = 89.04 %.

Por subconjunto, el LSTM–GAN mejoró el F1 en `logdb` (+6.95), `loggit` (+13.60) y `logsapp` (+1.03). Redujo la FPR en `logdb` (−1.39) y, sobre todo, en `logsapp` (−9.53, de 12.49 % a 2.95 %). En `loggit`, en cambio, aumentó la FPR (+0.76) y redujo el AUC (−8.94).

El F1 de `logsapp` es bajo para todos los modelos (< 3 %) porque el tramo de prueba contiene solo 8 ventanas maliciosas entre 8 860. Con un umbral en el percentil 95, cualquier modelo emite cientos de alertas por cada verdadero positivo.

5.2. Contraste de hipótesis

La Tabla 5 muestra los contrastes de Wilcoxon. H1 se confirma: el LSTM–GAN aumentó el F1 de forma significativa en el consolidado y en los tres subconjuntos. H2 se confirma en el consolidado, en `logdb` y en `logsapp`, aunque en `loggit` la FPR aumentó significativamente. H3 no se confirma, porque la diferencia de AUC no fue significativa en el consolidado ($p = .529$) y fue desfavorable en `loggit`.

El *bootstrap* a nivel de ventana confirma el patrón mixto del AUC:

- `logdb`: diferencia de +1.71 puntos, IC 95 % [−0.57, 4.07].
- `loggit`: diferencia de −8.87 puntos, IC 95 % [−19.85, −0.62].
- `logsapp`: diferencia de +8.36 puntos, IC 95 % [2.21, 15.48].

Las curvas ROC se presentan en la Figura 3.

5.3. Análisis de ablación y sensibilidad

Al retirar el discriminador (LSTM-AE), no hubo diferencias significativas en F1 ($p = .129$) ni en FPR ($p = .311$). El AUC, en cambio, mejoró en 3.36 puntos ($p < .001$, $r = .92$) y el MCC también fue mayor ($p = .002$). Además, el LSTM–GAN fue más inestable: en `logsapp`, la desviación de su AUC fue de 7.51 puntos frente a 0.36 del LSTM-AE.

Al elevar el umbral al percentil 99, el LSTM–GAN mantuvo un F1 de 33.68 % en `logdb`, con FPR = 1.69 %, mientras que Isolation Forest descendió a 19.96 %. En `loggit`, el LSTM–GAN obtuvo 44.39 % frente a 30.60 % de Isolation Forest. Su ventaja, por tanto, no depende de un umbral particular.

Tabla 4

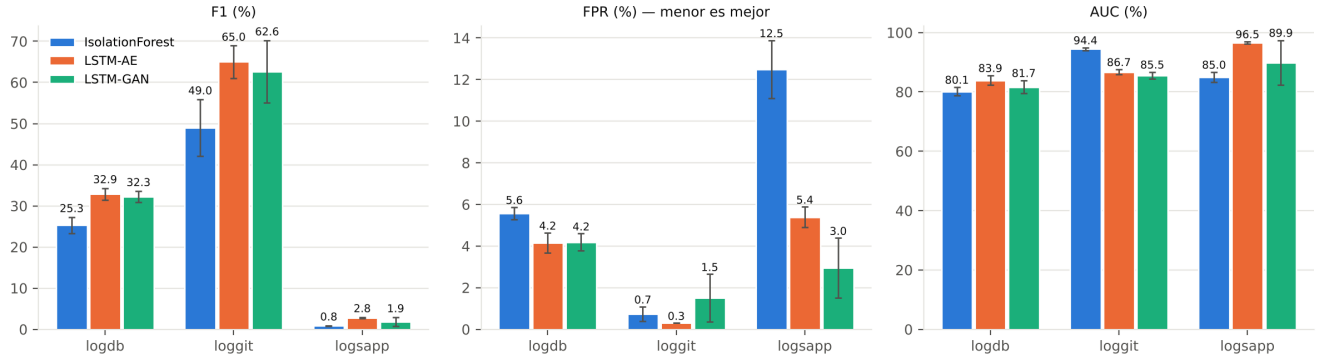
Indicadores de detección en el tramo de prueba (media ± desviación estándar de 10 semillas)

Subconjunto	Modelo	F1 (%)	Precisión (%)	Exhaustividad (%)	FPR (%)	AUC (%)	MCC
`logdb`	Isolation Forest	25.33 ± 1.96	26.86	24.00	5.57 ± 0.30	80.15 ± 1.40	.19
	LSTM-AE	32.89 ± 1.44	37.70	29.22	4.16 ± 0.49	83.88 ± 1.63	.28
	LSTM–GAN	32.28 ± 1.34	37.02	28.67	4.18 ± 0.42	81.67 ± 2.17	.28
`loggit`	Isolation Forest	49.03 ± 6.88	64.30	40.00	0.74 ± 0.35	94.44 ± 0.43	.49
	LSTM-AE	65.00 ± 3.96	84.65	52.86	0.30 ± 0.00	86.70 ± 0.80	.66
	LSTM–GAN	62.64 ± 7.61	63.71	65.24	1.50 ± 1.15	85.50 ± 1.18	.62
`logsapp`	Isolation Forest	0.82 ± 0.12	0.41	57.50	12.49 ± 1.40	84.97 ± 1.67	.04
	LSTM-AE	2.81 ± 0.15	1.43	86.25	5.39 ± 0.49	96.54 ± 0.36	.11
	LSTM–GAN	1.85 ± 1.09	0.96	33.75	2.95 ± 1.43	89.88 ± 7.51	.05
Consolidado	Isolation Forest	25.06	30.53	40.50	6.27	86.52	.24
	LSTM-AE	33.56	41.26	56.11	3.28	89.04	.35
	LSTM–GAN	32.26	33.90	42.55	2.88	85.68	.32

Nota. Umbral ρ en el percentil 95 de validación. FPR: menor es mejor.

Figura 2

F1, tasa de falsos positivos y AUC por modelo y subconjunto



Nota. Barras: media de 10 semillas; líneas de error: desviación estándar.

5.4. Hallazgos cualitativos

La Figura 4 muestra la puntuación S_t en el tramo de prueba. El modelo detectó, sin etiquetas, campañas reales de ataque:

- Inyecciones SQL ciegas basadas en tiempo (`sleep()`, `XOR`) contra PostgreSQL, el 1 y el 2 de mayo de 2026.
- Escaneos de *webshells* `.php` y de `/wp-login.php` contra GitLab, servicio que no ejecuta PHP.
- Sondeos de archivos `.env` y de `cgi-bin` contra el ERP.

Hubo también un caso que el modelo no detectó: una enumeración lenta de bases de datos el 8 de mayo, con unos 630 intentos repartidos en cinco horas. Superó el umbral al inicio, pero el contexto de 100 minutos la absorbió como comportamiento habitual.

6. Discusión

Los resultados respaldan parcialmente la hipótesis general. Coinciden con Gong et al. (2023) y Qian et al. (2025) en que la reconstrucción temporal de series latentes supera a los métodos estáticos en exactitud. La mejora observada, de 7.2 puntos de F1 y 54 % menos falsas alarmas, se explica porque Isolation Forest evalúa cada ventana de forma aislada, mientras que el LSTM interpreta el volumen y la composición de los eventos dentro de su contexto de 100 minutos. Esto es especialmente relevante en `logsapp`, cuyo tráfico de *longpolling* varía a lo largo del día.

Sin embargo, a diferencia de Gong et al. (2023), el componente adversarial no aportó valor sobre el autocodificador aislado e introdujo inestabilidad. Esto concuerda con las dificultades de entrenamiento de las GAN, como el desvanecimiento del gradiente y el colapso de modos, señaladas por Qian et al. (2025). Una explicación plausible es que la con-

Figura 3

Curvas ROC en el tramo de prueba (puntuaciones promediadas sobre 10 semillas)

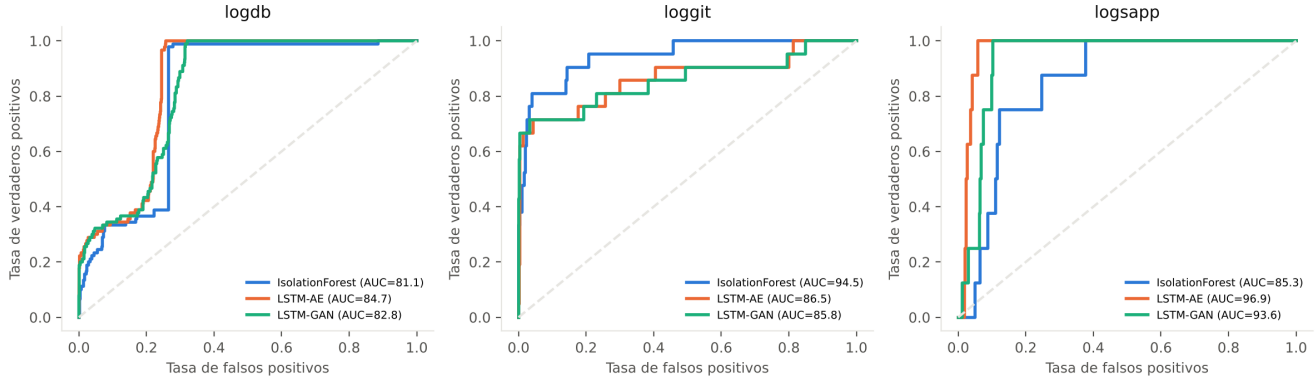


Tabla 5

Contraste de hipótesis: LSTM–GAN frente a Isolation Forest (prueba de Wilcoxon)

Hip.	Ámbito	$\bar{\Delta}$	p	r
H1 (F1)	Consolidado (n = 30)	+7.19	< .001	.85
	`logdb` / `loggit` / `logsapp`	+ / + / +	.002 / .002 / .027	
H2 (FPR)	Consolidado (n = 30)	−3.39	.002	.58
	`logdb` / `loggit` / `logsapp`	− / + / −	.002 / .035 / .002	
H3 (AUC)	Consolidado (n = 30)	−0.84	.529	.12
	`logdb` / `loggit` / `logsapp`	+ / − / +	.106 / .002 / .160	

Nota. $\bar{\Delta}$ = media de la diferencia LSTM–GAN menos Isolation Forest, en puntos porcentuales. Los signos por subconjunto indican la dirección del cambio; n = 10 pares en cada uno.

taminación del entrenamiento con ataques reales, ausente en los datos de sensores de esos estudios, lleva al discriminador a aprender también patrones maliciosos como reales.

La ventaja de Isolation Forest en el AUC de `loggit` confirma la advertencia de Landauer et al. (2023): métodos simples pueden competir con el aprendizaje profundo cuando las anomalías se manifiestan como cambios de frecuencia, como ocurre con las ráfagas de escaneo web. Del mismo modo, los valores de F1 obtenidos, entre 1.9 y 62.6 %, son muy inferiores a los publicados sobre conjuntos de referencia, por ejemplo el 99.61 % de Nasirzadeh et al. (2025) o el 97.3 % de AUC de Gorokhov et al. (2023) en HDFS. Esta brecha es coherente con Albert (2025), quien mostró que la partición cronológica y la presencia de anomalías en el entrenamiento reducen drásticamente el desempeño reportado, y es comparable al F1 medio de 71.23 % de Zhao et al. (2025) en escenarios de transferencia entre sistemas.

En un contexto más amplio, el hallazgo de mayor valor operativo es la reducción de falsas alarmas sin etiquetas. En `logsapp` equivale a unas 840 alertas falsas menos durante el periodo de prueba de 30 días, lo que alivia directamente la carga de un equipo de seguridad universitario. La omisión de la enumeración lenta ilustra el límite de los modelos de contexto fijo ante anomalías colectivas de baja intensidad (Zamanzadeh Darban et al., 2024). A ello se suma la sensibilidad al ruido de los registros reales, documentada por Zhang et al. (2025).

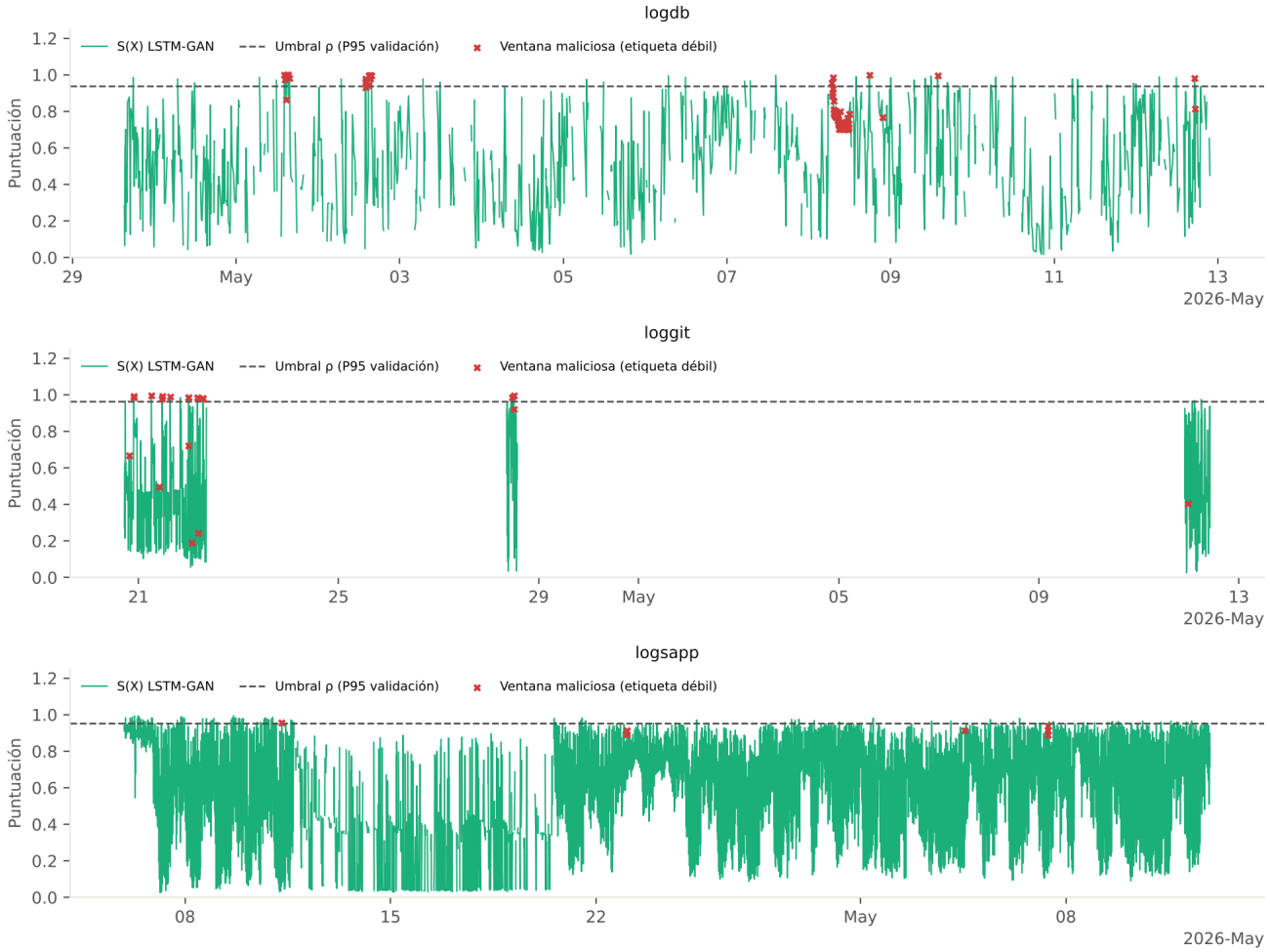
Limitaciones. Las etiquetas débiles, derivadas de reglas, no equivalen a una verdad de campo y pueden omitir ataques no contemplados en ellas. Se analizaron tres de los cuatro subconjuntos del repositorio. `logsapp` cuenta con muy pocas ventanas maliciosas en el tramo de prueba. Por último, no se realizó una búsqueda de hiperparámetros para evitar ajustar el modelo sobre el tramo de evaluación.

Líneas futuras. Se proponen cinco:

1. Calibrar ρ según la tasa esperada de anomalías de cada servicio (Qian et al., 2025).
2. Emplear contextos multiescala para capturar ataques lentos.
3. Incorporar explicabilidad para el analista (Landauer et al., 2023).
4. Transferir el modelo entre subsistemas mediante pseudoetiquetado (Zhao et al., 2025).
5. Fusionar las modalidades semántica y secuencial con transformadores colaborativos (Nasirzadeh et al., 2025).

Figura 4

Puntuación S_t del LSTM–GAN, umbral ρ y ventanas maliciosas en el tramo de prueba



Nota. Las cruces marcan ventanas con etiqueta débil maliciosa. Los huecos corresponden a periodos sin registros.

7. Conclusiones

Se cumplió el propósito de evaluar el efecto del modelo de agentes adversarios duales LSTM–GAN sobre la detección de comportamientos maliciosos sin etiquetas en registros de servidores universitarios. Las hipótesis específicas se verificaron como sigue:

- **H1 se valida.** El F1 aumentó de 25.06 % a 32.26 % ($p < .001$).
- **H2 se valida.** La FPR se redujo de 6.27 % a 2.88 % ($p = .002$).
- **H3 se rechaza.** El AUC no cambió de forma significativa ($p = .529$).

En consecuencia, la hipótesis general se acepta de forma parcial. La relevancia y novedad del estudio residen en evaluar este tipo de arquitectura sobre 3.2 millones de eventos crudos de tres servicios reales, con partición cronológica, entrenamiento contaminado y contraste estadístico. Se trata de condiciones más cercanas a la producción que las de los conjuntos de referencia habituales.

Los aportes principales son tres: una representación de ventanas como series semánticas que permite detectar ráfagas de ataque repetitivas, la evidencia mediante ablación de que la reconstrucción temporal y no el discriminador explica la mejora, y un repositorio reproducible. En cuanto a la aplicabilidad práctica, un equipo de seguridad universitario puede desplegar el modelo sin etiquetar datos y obtener menos de la mitad de falsas alarmas que con la técnica tradicional. Esto responde a la motivación inicial: aprovechar los abundantes registros institucionales sin depender de analistas para etiquetarlos.

Dataset y código fuente

- **Datos crudos:** subconjuntos `logdb`, `loggit` y `logsapp` del repositorio *uni-logs* (https://uni-logs.us-mia-1.linodeobjects.com/list.html).
- **Código y resultados:** los guiones `prep.py`, `experiment.py` y `analyze.py`, el archivo `requirements.txt`, las tablas de resultados y las figuras están disponibles en https://github.com/Arthur100803/ia2-tema7-agentes-adversarios-duales.
- **Ejecución completa:** los comandos están en el archivo `README.md`. Toma unos 105 minutos en una CPU de dos núcleos.

Referencias

Albert, R.-G. (2025). System logs anomaly detection. Are we on the right path? *Applied Artificial Intelligence*, *39*(1), Artículo e2440692. https://doi.org/10.1080/08839514.2024.2440692

Almodovar, C., Sabrina, F., Karimi, S., & Azad, S. (2024). LogFiT: Log anomaly detection using fine-tuned language models. *IEEE Transactions on Network and Service Management*, *21*(2), 1715-1723. https://doi.org/10.1109/TNSM.2024.3358730

Aslam, M. M., Tufail, A., De Silva, L. C., Apong, R. A. A. H. M., & Namoun, A. (2024). An improved autoencoder-based approach for anomaly detection in industrial control systems. *Systems Science & Control Engineering*, *12*(1), Artículo 2334303. https://doi.org/10.1080/21642583.2024.2334303

Gong, Z., Liu, Q., Ding, J., Wang, X., & Wang, P. (2023). A novel multi-dimensional time-series data anomaly detection model based on generative adversarial network aided autoencoder. *IFAC-PapersOnLine*, *56*(2), 8085-8090. https://doi.org/10.1016/j.ifacol.2023.10.960

Gorokhov, O., Petrovskiy, M., Mashechkin, I., & Kazachuk, M. (2023). Fuzzy CNN autoencoder for unsupervised anomaly detection in log data. *Mathematics*, *11*(18), Artículo 3995. https://doi.org/10.3390/math11183995

Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, *12*, Artículo 100470. https://doi.org/10.1016/j.mlwa.2023.100470

Nasirzadeh, M., Tahmoresnezhad, J., & Rashidi-Khazaee, P. (2025). A unified framework for detecting point and collective anomalies in operating system logs via collaborative transformers. *Scientific Reports*, *15*, Artículo 45698. https://doi.org/10.1038/s41598-025-27693-4

Qian, C., Tang, W., & Wang, Y. (2025). RGAnomaly: Data reconstruction-based generative adversarial networks for multivariate time series anomaly detection in the Internet of Things. *Future Generation Computer Systems*, *167*, Artículo 107751. https://doi.org/10.1016/j.future.2025.107751

Zamanzadeh Darban, Z., Webb, G. I., Pan, S., Aggarwal, C., & Salehi, M. (2024). Deep learning for time series anomaly detection: A survey. *ACM Computing Surveys*, *57*(1), 1-42. https://doi.org/10.1145/3691338

Zhang, H., Zhou, Y., Xu, H., Shi, J., Lin, X., & Gao, Y. (2025). Anomaly detection in virtual machine logs against irrelevant attribute interference. *PLOS ONE*, *20*(1), Artículo e0315897. https://doi.org/10.1371/journal.pone.0315897

Zhao, X., Guo, K., Huang, M., Qiu, S., & Lu, L. (2025). ELFA-Log: Cross-system log anomaly detection via enhanced pseudo-labeling and feature alignment. *Computers*, *14*(7), Artículo 272. https://doi.org/10.3390/computers14070272

Apéndice A
Matriz de operacionalización de variables

Tabla A1

Matriz de operacionalización

Variable	Dimensión	Indicador	Definición operacional / fórmula	Escala
VI: Modelo de agentes adversarios duales LSTM–GAN	D1. Reconstrucción temporal	Error de reconstrucción ($\mathcal{L}_{rec}$)	Error cuadrático medio, ecuación (4), en validación	Razón
		Épocas hasta convergencia	Épocas hasta la parada temprana (paciencia 5)	Razón
	D2. Discriminación adversarial	Pérdida del discriminador ($\mathcal{L}_D$)	Ecuación (5)	Razón
		Probabilidad de rechazo	$1 - D(\tilde{H}_t)$ por ventana	Razón [0,1]
	D3. Puntuación dual	Puntuación de anomalía S_t	Ecuación (7)	Razón [0,1]
		Tasa de alerta	Proporción de ventanas con $S_t > \rho$	Razón (%)
VD: Detección de comportamientos maliciosos	D1. Exactitud de la detección	Puntaje F1	$2PR/(P + R)$	Razón (%)
		Precisión (P) y exhaustividad (R)	$TP/(TP + FP)$; $TP/(TP + FN)$	Razón (%)
	D2. Tasa de falsas alarmas	Tasa de falsos positivos (FPR)	$FP/(FP + TN)$	Razón (%)
		Especificidad	$TN/(TN + FP) = 1 - \text{FPR}$	Razón (%)
	D3. Capacidad discriminante	Área bajo la curva ROC (AUC)	Área bajo la curva TPR–FPR	Razón (%)
		Coeficiente de Matthews (MCC)	Correlación entre la clase real y la predicha	Intervalo [−1,1]

Nota. Instrumentos: guion `experiment.py` (VI) y matriz de confusión frente a las etiquetas débiles del Apéndice C (VD).

Apéndice B
Matriz de consistencia

Tabla B1

Matriz de consistencia

Problema	Objetivo	Hipótesis	Variables y dimensiones	Metodología
General: ¿En qué medida el modelo LSTM–GAN mejora la detección de comportamientos maliciosos sin etiquetas en registros de servidores universitarios?	**General:** Evaluar en qué medida el modelo LSTM–GAN mejora dicha detección frente a una técnica no supervisada tradicional.	**General:** El modelo LSTM–GAN mejora significativamente la detección de comportamientos maliciosos.	**VI:** Modelo LSTM–GAN (D1 reconstrucción temporal, D2 discriminación adversarial, D3 puntuación dual).	**Enfoque:** cuantitativo. **Tipo:** aplicado. **Nivel:** explicativo. **Diseño:** cuasi-experimental pre-prueba/post-prueba con ablación.
PE1: ¿En qué medida mejora la exactitud de la detección?	**OE1:** Determinar el efecto sobre el puntaje F1.	**H1:** El LSTM–GAN aumenta significativamente el F1.	**VD:** Detección de comportamientos maliciosos (D1 exactitud).	**Población:** eventos del repositorio *uni-logs*. **Muestra:** 53 416 ventanas de 5 min (3 subconjuntos), por conveniencia.
PE2: ¿En qué medida reduce la tasa de falsas alarmas?	**OE2:** Determinar el efecto sobre la FPR.	**H2:** El LSTM–GAN reduce significativamente la FPR.	(D2 tasa de falsas alarmas)	**Técnica:** análisis documental. **Instrumentos:** Drain3, ficha de reglas (Apéndice C), guiones Python.
PE3: ¿En qué medida mejora la capacidad discriminante?	**OE3:** Determinar el efecto sobre el AUC-ROC.	**H3:** El LSTM–GAN aumenta significativamente el AUC.	(D3 capacidad discriminante)	**Análisis:** estadística descriptiva; Wilcoxon pareado (α = .05); *bootstrap* del AUC.

Apéndice C
Instrumento de recolección de datos

C.1 Ficha de etiquetado débil por ventana. Para cada ventana de cinco minutos se registran los campos `win` (inicio, UTC), `n_events` (número de eventos), el conteo de cada regla de la Tabla 2 (`sqli`, `probe`, `web`, `scan`, `ssh`, `c4xx`) y la etiqueta $y \in \{0, 1\}$. El instrumento se aplica automáticamente con `prep.py` y su salida queda en los archivos `<subconjunto>_windows.csv`.

C.2 Ficha de registro de resultados por corrida. Para cada combinación de subconjunto, semilla y modelo se registran los campos `epochs`, F1, precisión, exhaustividad, FPR, AUC, MCC, TP, FP, TN y FN. La salida queda en los archivos `results_runs_*.csv`.

C.3 Validez y confiabilidad. Las reglas se basan en indicadores de compromiso inequívocos en el contexto analizado: por ejemplo, las peticiones a `.php` carecen de uso legítimo en GitLab y en Odoo. La confiabilidad se garantiza porque el instrumento es determinista: la misma entrada produce siempre las mismas etiquetas.